

**EXERCISE 10.9**

Hard: Use `foldMap` to detect whether a given `IndexedSeq[Int]` is ordered. You'll need to come up with a creative `Monoid`.

10.4 Example: Parallel parsing

As a nontrivial use case, let's say that we wanted to count the number of words in a `String`. This is a fairly simple parsing problem. We could scan the string character by character, looking for whitespace and counting up the number of runs of consecutive nonwhitespace characters. Parsing sequentially like that, the parser state could be as simple as tracking whether the last character seen was a whitespace.

But imagine doing this not for just a short string, but an enormous text file, possibly too big to fit in memory on a single machine. It would be nice if we could work with chunks of the file in parallel. The strategy would be to split the file into manageable chunks, process several chunks in parallel, and then combine the results. In that case, the parser state needs to be slightly more complicated, and we need to be able to combine intermediate results regardless of whether the section we're looking at is at the beginning, middle, or end of the file. In other words, we want the combining operation to be associative.

To keep things simple and concrete, let's consider a short string and pretend it's a large file:

```
"lorem ipsum dolor sit amet, "
```

If we split this string roughly in half, we might split it in the middle of a word. In the case of our string, that would yield `"lorem ipsum do"` and `"lor sit amet, "`. When we add up the results of counting the words in these strings, we want to avoid double-counting the word `dolor`. Clearly, just counting the words as an `Int` isn't sufficient. We need to find a data structure that can handle partial results like the half words `do` and `lor`, and can track the complete words seen so far, like `ipsum`, `sit`, and `amet`.

The partial result of the word count could be represented by an algebraic data type:

```
sealed trait WC
case class Stub(chars: String) extends WC
case class Part(lStub: String, words: Int, rStub: String) extends WC
```

A `Stub` is the simplest case, where we haven't seen any complete words yet. But a `Part` keeps the number of complete words we've seen so far, in `words`. The value `lStub` holds any partial word we've seen to the left of those words, and `rStub` holds any partial word on the right.

For example, counting over the string "lorem ipsum do" would result in `Part("lorem", 1, "do")` since there's one certainly complete word, "ipsum". And since there's no whitespace to the left of `lorem` or right of `do`, we can't be sure if they're complete words, so we don't count them yet. Counting over "lor sit amet, " would result in `Part("lor", 2, "")`.



EXERCISE 10.10

Write a monoid instance for `WC` and make sure that it meets the monoid laws.

```
val wcMonoid: Monoid[WC]
```



EXERCISE 10.11

Use the `WC` monoid to implement a function that counts words in a `String` by recursively splitting it into substrings and counting the words in those substrings.

Monoid homomorphisms

If you have your law-discovering cap on while reading this chapter, you may notice that there's a law that holds for some functions *between* monoids. Take the `String` concatenation monoid and the integer addition monoid. If you take the lengths of two strings and add them up, it's the same as taking the length of the concatenation of those two strings:

```
"foo".length + "bar".length == ("foo" + "bar").length
```

Here, `length` is a function from `String` to `Int` that *preserves the monoid structure*. Such a function is called a *monoid homomorphism*.⁶ A monoid homomorphism `f` between monoids `M` and `N` obeys the following general law for all values `x` and `y`:

```
M.op(f(x), f(y)) == f(N.op(x, y))
```

The same law should hold for the homomorphism from `String` to `WC` in the present exercise.

This property can be useful when designing your own libraries. If two types that your library uses are monoids, and there exist functions between them, it's a good idea to think about whether those functions are expected to preserve the monoid structure and to check the monoid homomorphism law with automated tests.

⁶ *Homomorphism* comes from Greek, *homo* meaning "same" and *morphe* meaning "shape."